

CS 12 Cheatsheet

Diogn Lei R. Mortera

202508438 · Seat 81 · TCD/FRU2

Commands

```
#pyright : strict
from __future__ import annotations
from dataclasses import dataclass
from enum import Enum, auto
if __name__ == '__main__': main()
```

git push, pull, commit -m '<msg>', add

pytest coverage run --batch -m pytest, coverage html

pyxel pyxel run <file>, pyxel package . <fname>.py, pyxel app2html <fname>.pyxapp

Introductory Concepts

type hinting (:,>), **inference** (**Pyright**), **narrowing**, **aliases** sub/supertypes, long-lived programs, exhaustiveness checking

model-view-controller/presenter model - game logic, view - display logic (terminal, pyxel), controller - unify model and view, game loop

pre/postconditions conditions before/after function is called

invariant condition that is true throughout entire lifetime of object (and by LSP, its subclasses)

inversion of control flow of control given to code by framework (e.g. food service that delivers healthy food to house every week)

search space reduction split code into units for easier unit testing

Testing and Debugging

input/output equivalence partitions, edge cases, boundary values, search space reduction, unit/stress/fuzz testing, pytest

non-throwaway code bug-free, easy to understand, easy to change; desired outcome of testing

test coverage % parts of code executed by **test suite**, 100% coverage $\neq$ no bugs

line/branch coverage lines / t/f conditionals in assembly code

Modularization

module, modularization smaller, reusable components

separation of concerns nonoverlapping aspects of code (related (but not) S in SOLID)

dependency baked-in (hard coded), injected (e.g. Strategy patterns), decoupled code easier to change

SOLID

single responsibility separation of concerns, gathering of things that change for same reason

open closed open for extension, closed for modification

Liskov substitution behavioral subtyping, history rule, pre/postconditions, Barbara Liskov

interface segregation can separate interfaces, implicit (Protocol) and explicit (ABC / abstract base class)

dependency inversion depend on interfaces and not concrete classes

Subtyping

subtype polymorphism, hierarchical substitutability of types

subtyping hierarchy ABC, exception, numeric tower

from collections.abc from collections

covariance subtype of complex follows simple

contravariance subtype of complex is the reverse of simple

bivariance either direction applies

invariance no trend

Basic Subtyping

```
<: - preorder, reflexive, transitive
A <: A
A <: A | B, B <: A | B
A <: B and B <: C -> A <: C #transitive
A <: object
```

Container Subtyping

```
Drink <: Liquid
ROL[Drink] <: ROL[Liquid] #read-only cov
WOL[Liquid] <: WOL[Drink] #write-only contraV
```

Functional Subtyping

Number = float | int #let

```
perform_operation(f : Callable[[Number],
Number]):
    return f(a,b)
```

```
Let Num = int | float
int <: Num
```

```
f(x : Num) <: f(x : int) #parameter
contravariance
f -> int <: f -> Num #return covariance
```

LSP Rules

contravariant method parameters and **covariant return types**

exceptions thrown subtypes of original (e.g. if class has no exception, then its subclass must have none too)

no strengthening preconditions don't require stricter than originally needed (e.g. if superclass demands ≥ 0 , subclass should not demand > 0)

no weakening postconditions (e.g. if superclass provides nonempty deck, subclass must not provide empty deck)

no violating invariant must not violate behavior based on specified invariants (in docstrings), including interactions and calling (or not calling) exceptions, staying true to roots

no violating history rule any state transition of child has to be possible using parent's methods (e.g. if you can create some method that uses other methods of a parent to perform that task)

Object-Oriented Programming

(networks of message-sending objects)

abstraction ABCs, explicit interface, implementation hidden

encapsulation one concern - one responsibility, each object featuring attributes and methods that change properties of that class

polymorphism subtype polymorphism, subclasses

inheritance from ABCs or instantiated classes to create derived classes

composition objects referencing other objects; dependency injection

static method, static factory method

forwarding passing message to another object

more objects, more cohesive yet complex

maximize cohesion, minimize coupling

public interface @property, non _-methods

Note: _-prefixed vars are *protected*, NOT *private* (e.g. can't be accessed except by children)

OODP

object-oriented design patterns

creational abstract factory, builder, factory method, prototype, singleton

structural *adapter*, bridge, composite, decorator, facade, flyweight, proxy

behavioral chain of responsibility, command, iterator, interpreter, mediator, memento, *observer*, *state*, *strategy*, *template method*, visitor

Structural

adapter incompatible (contained in adapter) + compatible interface (implemented in adapter) + adapter class (conversion logic)

Behavioral

observer allows observable to notify observers without knowing who observers are

- Observer = Subscriber = Listener
- Observable = Publisher

state sole purpose is to change the state (good with `itertools.cycle`), Enum-based entries has `next(self)` function

strategy introduces behavior via composition, has (a) strategy interface (e.g. `GuessingGameStrategy`), (b) concrete class (e.g. `BinarySearchStrategy`), and (c) context class (e.g. `Player(s : Strategy)`)

template method delegates implementation of *some* steps to subclasses, usually ABCs

Game Programming

graphical, nonblocking I/O, game loop, inversion of control, FPS, normalize speed by dividing by FPS, use MVC

Essentials

```
pyxel.init(size_x, size_y, fps=fps)
def update(): ... # game state
def draw(): ...# screen
pyxel.run(update, draw)
```

.mouse(True)

Drawing

```
.text(x,y,text,color)
.circ(x,y,r,color)
.rect/.rectb(x,y,w,h,color)
.cls(color)
.mouse_x .mouse_y .mouse_wheel
KEY_[CHAR]{NUMBER}{UP/DOWN/LEFT/RIGHT}
KEY_RETURN
```

Running

```
pyxel run <file> | pyxel package . <fname>.py |
pyxel app2html <fname>.pyxapp
```

HTML / DOM

HyperText Markup Language (HTML) defines how web pages are structured

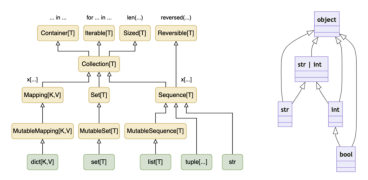
document object model (DOM) a tree representing the structure of an HTML page in tree form

open-close / self-closing tags, inline (take necessary space) and block (takes entire page width) elements

scripting language to manipulate the DOM tree (e.g. JS, TypeScript, PyScript)

```
<html><head><title><body>
<p><div><h1/2/..6> <!-- block elements -->
<span><input> <!-- inline elements -->
document.createElement()
.getElementById('#id') .querySelector('.class')
.textContent .innerHTML
```

Types

[illegible]

```

graph TD
    window[window] --> document[document]
    document --> HTML[HTML]
    HTML --> head[head]
    HTML --> body[body]
    head --> meta1[meta]
    head --> meta2[meta]
    head --> title[title]
    head --> link[link]
    body --> div1[div]
    body --> table[table]
    body --> p[p]
    div1 --> img[img]
    div1 --> div2[div]
    div1 --> div3[div]
    table --> tr1[tr]
    table --> tr2[tr]
    table --> tr3[tr]
    p --> p1[p]
  
```

Diagram illustrating the hierarchical structure of a document object model (DOM) tree. The root node is 'window' (object), which points to 'document' (object), which points to 'HTML' (object). The 'HTML' node has two children: 'head' and 'body'. The 'head' node has four children: 'meta', 'meta', 'title', and 'link'. The 'body' node has three children: 'div', 'table', and 'p'. The 'div' node has three children: 'img', 'div', and 'div'. The 'table' node has three children: 'tr', 'tr', and 'tr'. The 'p' node has one child: 'p'. The diagram also shows the relationship between the nodes and the document object model (DOM) tree structure.

```

classDiagram
    class Customer {
        name: String
        email: String
        phone: String
        password: String
    }
    class Order {
        id: Integer
        date: String
        items: List
        status: String
        user: User
    }
    class Product {
        name: String
        price: Integer
        stock: Integer
        category: String
    }
    class Item {
        productId: Integer
        productId: String
        productId: Integer
        productId: String
    }
    class Payment {
        amount: Integer
        type: String
    }
    class Cart {
        productId: Integer
        productId: String
        productId: Integer
        productId: String
    }
    class User {
        name: String
        email: String
        password: String
    }

    Customer "1" -- "0..*" Order : Multiplicity
    Order "1" -- "0..*" Product : Aggregation
    Order "1" -- "0..*" Item : Role
    Order "1" -- "0..*" Payment : Operation
    Order "1" -- "0..*" Cart : Operation
    Order "1" -- "0..*" User : Operation
    
```

The diagram illustrates the relationships between various entities in an online store. The **Customer** class is associated with the **Order** class, with a multiplicity of 1 for the customer and 0..* for the order. The **Order** class is associated with the **Product** class, with a multiplicity of 1 for the order and 0..* for the product. The **Order** class is also associated with the **Item** class, with a multiplicity of 1 for the order and 0..* for the item. The **Order** class is associated with the **Payment** class, with a multiplicity of 1 for the order and 0..* for the payment. The **Order** class is associated with the **Cart** class, with a multiplicity of 1 for the order and 0..* for the cart. The **Order** class is associated with the **User** class, with a multiplicity of 1 for the order and 0..* for the user.

```
model.py | view.py | controller.py
controller =
CalcController(CalcModel(),CalcView())
# put most print() / draw() in V
# put most logic in M
# call and pass methods between M & V in C
```

```
bool | int | str | Literal['mkns', 'mkbn'] |
list[int] | dict[int, str] | Callable[[int],
int]

tuple(1,2,3) : tuple[int,int,int]

tuple(varying size) : tuple[int, ...]

Grid = list[list[str]] # type alias

# f(g(x)) type-checks if return(g) <:
param_type(f)

#ILLEGAL

list[int, ...] #... only for immutable
containers (e.g. tuple[int, ...])
```

```
ROL[Drink] <-: ROL[Liquid] # drink can do
anything a liquid can (history rule) |
COVARIANCE

ROL[Liquid] <-: ROL[Drink] # not all liquids
can be drank

WOL[Liquid] <-: WOL[Drink] # only cares if
things inside are its subtypes |
CONTRAVARIANCE
```

```
f (x : Number) <=: f (x : int) #let addition
be an f(x : Number). type safety maintained
because int's can be added as it is a subtype
of Number

f (x : int) <=: f (x : Number) #let modulo be
an f(x : int). then some numbers cannot be
modulo-ed (like floats)
```

```
f -> int <: f -> Number # an int can still
perform everything expected of a Number (e.g.
+, -, *)
f -> Number <:/: f -> int
# will 'return list[f]' still work if f ->
Number, which can be a float?
```

- making types of subclasses parameters more specific
- subclasses override superclass
- errors not carried over in subclasses
- different / unpassed parameters from sub/superclass
- invariant being overridden
- inadvertent behaviors despite invariant statement
- narrowing parameter types in overridden methods
- name mangling** issues
- forgetting `super().__init__()` . . .

Strategy

```
class Human:
    def __init__(love_language :
LoveLanguage): ...
class LoveLanguage(Protocol): ...
# classes below realizes interface above
class TimeLoveLanguage: ...
class TouchLoveLanguage: ...

time_lover = Human(TimeLoveLanguage)
touch_lover = Human(TouchLoveLanguage)
```

```
LoveLanguageKind = Enum('LoveLanguageKind',
    'TIME TOUCH')

class LoveLanguageFactory:
    @classmethod
    def make(cls, ll_kind : LoveLanguageKind):
        match ll_kind:
            case LoveLanguageKind.TIME:
                love_language = TimeLoveLanguage()
            case LoveLanguageKind.TOUCH:
                love_language = TouchLoveLanguage()

        return Human(love_language)
```

```
class Biscuit(ABC):
    def __init__(self):
    def take(self): print('Take it')

    @abstractmethod
    def break(self): ...

    @abstractmethod
    def taste(self): ...

class Cookie(Biscuit):
    def break(self): print("Don't break it")
    def taste(self): print("I wanna see you
taste it")
```

```
class Feeling():
    def __init__(self):
        self._state = HappyState(self)

    def change_state(self, state :
FeelingState):
        self._state = state

    def next(self):
        self._state = self._state.next()
        return self._state

class FeelingState(ABC):
    def __init__(self, f : feeling):
        self._feeling = f

    def set_state(self, state : FeelingState):
        self._feeling.change_state(state)
    @abstractmethod
    def next(self): pass

class HappyState(FeelingState):
    def next(self):
        print('S')
```

Observer

```
# the observer "subscribes" to the
observable.

def follow(self, account :
SocialMediaAccount):
    self._follow(account)
    account.add_follower(self)

def add_follower(self, account :
SocialMediaAccount):
    self.add(account)

# the observable is updating all of its
"observers"

def update_followers(self, post : Post):
    for follower in self._followers:
        follower.update_feed(post)

def update_feed(self, post : Post):
    self._feed.append(post)
```

```
@dataclass
class Vector:
    magnitude: float
    direction: float # radians

@dataclass
class VectorComponent:
    x: float
    y: float

    def get_components(self) ->
VectorComponent: ...

class VectorToVectorComponentAdapter(
    VectorComponent):
    def __init__(self, vector: Vector):
        self._vector = vector

    def get_components(self) ->
VectorComponent:
    x = self._vector.magnitude *
math.cos(self._vector.direction)
    y = self._vector.magnitude *
math.sin(self._vector.direction)
    return VectorComponent(x, y)

v = Vector(50, math.pi / 2)
adapter = VectorToVectorComponentAdapter(v)

print( adapter.get_components() ) # outputs
around (0, 50)
```

```
import pygame
from dataclasses import dataclass

FPS = 60
SPEED = 10
DT = SPEED / FPS

class World: WIDTH = 200; HEIGHT = 200

@dataclass
class Ball:
    x: float; y: float; r: float; v_x:
    float; v_y: float; g: int = 10

    def update(self):
```

HTML / DOM Sample

```
from pyscript import document, web
#warning: better to use MVC, decreases
#global 'spam'

root = document.getElementById("app")

count = 0

def handle_incr(ev):
    global count
    print("+ was clicked")
    count += 1
    ctr.innerHTML = str(count)

def handle_decr(ev):
    global count
    print("- was clicked")
    count -= 1
    ctr.innerHTML = str(count)
```

Labs

Lab 01: CS11 Recall, Type Hinting, Lab 2: Battleship, Lab 3: Whac-A-Mole: Terminal Version
Lab 4: Lab 4: Connect-Tac-Toe - Lab 5: Berry Interesting Pokémon Battle

Instructors
Jeremy Curry, Beltran, David Carleton Ko, Miguel Martinez, Joelle Addave, Ren Tristan Dela Cruz, Mario Carreon, [Juan-Luis-Cameroi](#)

Student Assistants
Vaughan Agpino · Eliana Mari Lim · Charlie Sim · Gabby Sacramento · Aeron Du Puy · Pauladilla · Jovelyn Gail Riosaforte · Lucas Agacali · Marius Urbueso · Tristan Jovan Novak

Slide Titles
(0-indexed) **HH** · Static Type Checking (in Python) · Debugging and Testing · Separation of Concerns · Subtyping · Object-Oriented Programming and Design · Single Inheritance · Type Variance and Subtyping · Behavioral Subtyping · Behavioral Subtyping · Object-Oriented Design Patterns · Game Programming Basics · Game Programming II (MVC) · HTML and the DOM (PyScript)

Credits
• Diego Mortera (majority of helpset on code for OOPD and LSP)
• Jared Liston, Justin Canamio (chaseup on LSP checklist)
• Jerome Curry Beltran (pyscript code for Loe 13)
• Seowoon Krayn (Cat's behavior checklist for Double-Check)
• CS 12.25.1 and 25.2 Slides
• certain people for Motivational Quotes :)

Motivational Quotes
"miso ko na sya na"
"Take it, don't break it, I wanna see you taste it" *Cookie, Newsans*
"I'll push you" *my speech/s estimate*
"It can't be that hard" *gm*
"Ah, 'bi na malabo" **LASIK, HEY JUNE!**
"Truth emerges more readily from fear than confusion" *Thomas Kuhn, The Structure of Scientific Revolutions*
"am i gonna code or am i going to be coded?" *gm*
"If only makes the suffering that would befall us next..." **ULTRAKILL**
"What's a known a test good for? Well! The most basic and obvious answer to that question is that good tests describe what you want to measure, and bad tests do not." *Ben Clegg, The*
This is a Trick Question
"Reviewing is doubling your potential." *ngre*
"It hurts to be something, it's worse to be nothing with you" *Premiere, Lasik*
"I live 98 percent of my mental energy to CS12. Others give only 2 percent." *Babbargown*
"One who loses to the sweeping winds will sweep away themselves: only by opposing the breeze will one reach the treasure it guards so tightly" *Justin*
"Momentum without sustainability is death" *World Economic Forum*
"Cause there's something and there is nothing, there is something in between" *Ricky Montenegro, Lie Without a Hook*